

Отчёт №3 по прохождению курса Stepik

Продвинутые темы

Мухина Ксения Николаевна

Содержание

1	Цель работы	6
2	Задание	7
3	Теоретическое введение	8
4	Выполнение курса	9
5	Выводы	23
	Список литературы	24

Список иллюстраций

4.1 Тестовое задание.	9
4.2 Тестовое задание.	9
4.3 Тестовое задание.	10
4.4 Тестовое задание.	10
4.5 Тестовое задание.	11
4.6 Тестовое задание.	11
4.7 Тестовое задание.	11
4.8 Тестовое задание.	12
4.9 Тестовое задание.	12
4.10 Тестовое задание.	12
4.11 Тестовое задание.	13
4.12 Тестовое задание.	13
4.13 Тестовое задание.	13
4.14 Тестовое задание.	14
4.15 Тестовое задание.	14
4.16 Тестовое задание.	15
4.17 Тестовое задание.	15
4.18 Тестовое задание.	16
4.19 Тестовое задание.	16
4.20 Тестовое задание.	17
4.21 Тестовое задание.	17
4.22 Тестовое задание.	18
4.23 Тестовое задание.	18
4.24 Тестовое задание.	18
4.25 Тестовое задание.	19
4.26 Тестовое задание.	19
4.27 Тестовое задание.	19
4.28 Тестовое задание.	19
4.29 Тестовое задание.	20
4.30 Тестовое задание.	20
4.31 Тестовое задание.	20
4.32 Тестовое задание.	20
4.33 Тестовое задание.	21
4.34 Тестовое задание.	21
4.35 Тестовое задание.	21

4.36 Тестовое задание.	22
--------------------------------	----

Список таблиц

1 Цель работы

Пройти курс на платформе Stepik «Введение в Linux». Ознакомиться с операционной системой Linux и её функциями.

2 Задание

Просмотреть теоретический материал в формате видео и текста. На основе полученной информации пройти тестовые задания курса.

3 Теоретическое введение

OS Linux - семейство Unix-подобных операционных систем на базе ядра Linux, включающих тот или иной набор утилит и программ проекта GNU, и, возможно, другие компоненты. Как и ядро Linux, системы на его основе создаются и распространяются в соответствии с моделью разработки свободного и открытого программного обеспечения. Linux-системы распространяются в основном бесплатно в виде различных дистрибутивов в форме, готовой для установки и удобной для сопровождения и обновлений, и имеющих свой набор системных и прикладных компонентов, как свободных, так и проприетарных.

4 Выполнение курса

Следующие тестовые и практические задания не нуждаются в подробном пояснении.

Какую клавишу(и) нужно нажать на клавиатуре, чтобы выйти из редактора vim? Считайте, что вы только что открыли файл и вам сразу понадобилось выйти из редактора.

Выберите один вариант из списка

☒ Правильно.

- ☒ ".", затем "q", затем "Enter"
- ☐ "Ctrl", затем "x"
- ☐ "q", затем "Enter"
- ☐ "Esc"
- ☐ "Q"

Рисунок 4.1: Тестовое задание.

Выберите все подходящие ответы из списка

☒ Верно. Так держать!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ Чтобы попасть в конец строки, нужно совершить меньше нажатий на W, чем на w
- ☐ Нажимая только на w, нельзя переместить курсор на "."
- ☐ В этой строке 9 "больших слов" (WORD)
- ☐ В этой строке 12 "слов" (word)
- ☐ Чтобы попасть в конец строки, нужно одинаковое число нажатий, что на W, что на w
- ☐ В этой строке 5 "слов" (word)

Рисунок 4.2: Тестовое задание.

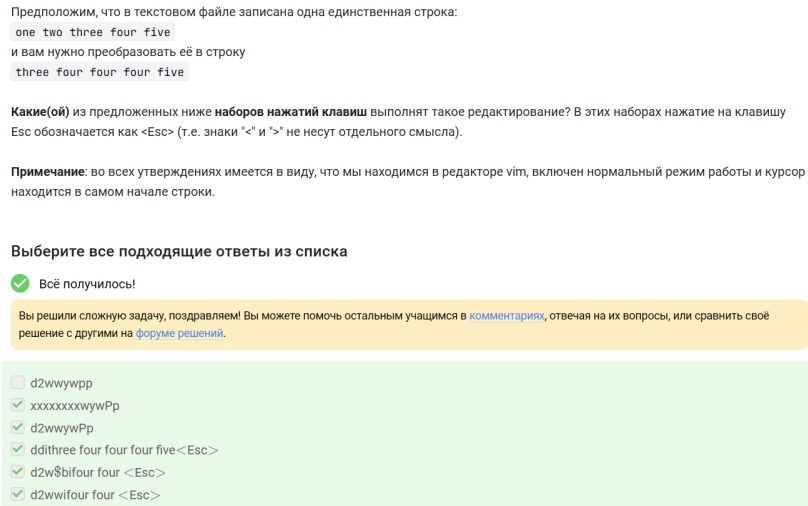


Рисунок 4.3: Тестовое задание.

Следующая команда используется для поиска и замены слов.

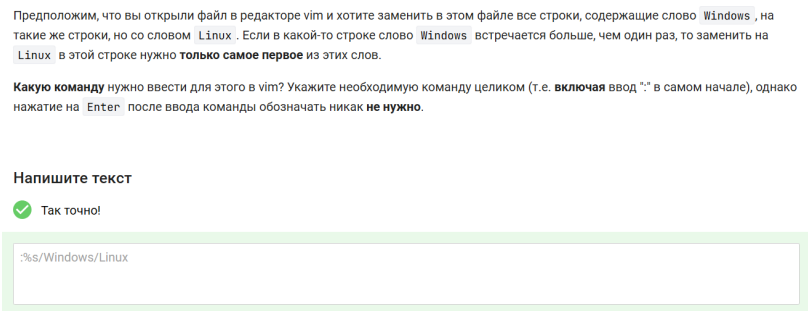


Рисунок 4.4: Тестовое задание.

Следующие тестовые и практические задания не нуждаются в подробном пояснении.

Мы совсем не рассказали вам про третий режим работы vim – режим **выделения (Visual)**. Предлагаем вам ознакомиться с ним самостоятельно. Например, это можно сделать во время прохождения упражнений в vimtutor, который мы настоятельно рекомендуем вам для изучения vim!

Чтобы убедиться, что вы разобрались с этим режимом работы, отметьте, пожалуйста, **все верные** утверждения из списка ниже.

Подсказка: если вы не хотите проходить vimtutor целиком, то можете открыть его и поиском найти слово **"Visual"**. Вы попадете в задание, прохождение которого будет вполне достаточно, чтобы выполнить это задание.

Выберите все подходящие ответы из списка

☒ Правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить свое решение с другими на [форуме решений](#).

- ☐ Чтобы выйти из режима выделения, нужно ввести :q
- ☒ Выйти из режима выделения можно, нажав клавишу Esc два раза
- ☒ В режиме выделения можно использовать команды d (удалить) и y (скопировать)
- ☒ В режиме выделения можно использовать команды перемещения (например, W, e, \$, и др.)
- ☒ Когда вы находитесь в режиме выделения, внизу редактора горит надпись – VISUAL – (или – ВИЗУАЛЬНЫЙ РЕЖИМ –)
- ☒ Режим выделения открывается из нормального режима по нажатию "v"

Рисунок 4.5: Тестовое задание.

Надеемся, что вы разобрались, что одну оболочку (например, `sh`) можно запустить из другой оболочки (например, из `bash`).

Предположим, что вы открыли терминал и у вас в нем запущена оболочка `bash`. Вы набираете в ней команды `A1`, `A2`, `A3`, а затем запускаете оболочку `sh`. В этой оболочке вы набираете команды `B1`, `B2`, `B3` и запускаете оболочку `bash`. И, наконец, в этой последней оболочке вы набираете команды `C1`, `C2`, `C3`. Если теперь вы попытаете при помощи стрелочек вверх/вниз перемещаться по истории набранных команд, то команды из какого набора(ов) будут появляться?

Выберите один вариант из списка

☒ Хорошие новости, верно!

- ☐ Никакие команды появляться не будут
- ☐ Из наборов A и C
- ☒ Только из набора C
- ☐ Только из набора B
- ☐ Из наборов B и C

Рисунок 4.6: Тестовое задание.

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [script1.sh](#), [script2.sh](#).

Предположим, что вы находитесь в директории `/home/bi/Documents/` и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash
cd /home/bi/
touch file1.txt
cd /home/bi/Desktop/
```

Как будет выглядеть **абсолютный путь** до созданного файла `file1.txt` по окончании работы скрипта?

Выберите один вариант из списка

☒ Всё получилось!

- ☐ Никак (файла `file1.txt` не будет существовать после завершения работы скрипта)
- ☐ `/home/bi/Desktop/file1.txt`
- ☒ `/home/bi/file1.txt`
- ☐ `/home/bi/Documents/file1.txt`

Рисунок 4.7: Тестовое задание.

Следующий вариант был выбран исходя из условий задания переменных:

не должно содержать символы вроде \$, @ и начинаться с цифры.

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [variables1.sh](#), [variables2.sh](#).

Какие из представленных ниже строк **могут** быть именами переменных в bash? Выберите **все** подходящие варианты!

Подсказка: если все варианты ответов являются неверными, то не отмечайте ни один из них и нажимайте кнопку "Отправить"/"Submit".

Выберите все подходящие ответы из списка

✔ Правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ variable123
- ☐ var@iabile
- ☐ 123variable
- ☐ var i able
- ☐ var-i-able
- ☐ variab\$\$le
- ☐ var.i.able

Рисунок 4.8: Тестовое задание.

Следующие тестовые и практические задания не нуждаются в подробном пояснении.

Напишите программу. Тестируется через stdin → stdout

✔ Отличное решение!

Теперь вам доступен [форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 var1=$1
2 var2=$2
3 var3=' $1 '
4 var4=' $2 '
5
6 echo "Arguments are: $var3=$1 $var4=$2"
```

Рисунок 4.9: Тестовое задание.

Выберите все подходящие ответы из списка

✔ Всё правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ -s \$0
- ☒ \$var1 == \$var2 || \$var1 != \$var2
- ☒ \$# -ge 0
- ☒ 5 -ge 5
- ☒ -z ""
- ☐ -z " "

Рисунок 4.10: Тестовое задание.

Какие строки и в какой последовательности он выведет на экран, если сначала этот скрипт запустили задав переменную `var=3`, а затем запустили еще раз, но уже с `var=5`.

Выберите один вариант из списка

☒ Всё получилось!

- ☐ Сначала four, потом one
- ☐ Сначала two, потом four
- ☒ Сначала four, потом four
- ☐ Сначала one, потом two

Рисунок 4.11: Тестовое задание.

Напишите программу. Тестируется через `stdin` → `stdout`

☒ Отличное решение!

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 var=$1
2
3 if [[ $var -eq 0 ]]
4 then
5   echo "No students"
6 elif [[ $var -eq 1 ]]
7 then
8   echo "1 student"
9 elif [[ $var -eq 2 ]]
10 then
11   echo "2 students"
12 elif [[ $var -eq 3 ]]
13 then
14   echo "3 students"
15 elif [[ $var -eq 4 ]]
16 then
17   echo "4 students"
18 else
19   echo "A lot of students"
20 fi
```

Рисунок 4.12: Тестовое задание.

Посмотрите на фрагмент `bash`-скрипта:

```
for str in a , b , c_d
do
  echo "start"
  if [[ $str > "c" ]]
  then
    continue
  fi
  echo "finish"
done
```

Если запустить этот скрипт, то **сколько раз** на экран будет выведено слово `"start"`, а сколько раз слово `"finish"`?

Выберите один вариант из списка

☒ Верно. Так держать!

- ☐ 5 раз "start" и 5 раз "finish"
- ☐ 3 раза "start" и 2 раза "finish"
- ☒ 5 раз "start" и 4 раза "finish"
- ☐ 3 раза "start" и ни разу "finish"

Рисунок 4.13: Тестовое задание.

```

1 while true
2 do
3     echo "enter your name:"
4     read name
5     if [[ -z $name ]]
6     then
7         echo "bye"
8         break
9     fi
10
11    echo "enter your age:"
12    read age
13    if [[ $age -le 0 ]]
14    then
15        echo "bye"
16        break
17    fi
18
19    case "$age" in
20        [1-9]|[10-6]) echo "$name, your group is child";;
21        1[7-9]|2[0-5]) echo "$name, your group is youth";;
22        *) echo "$name, your group is adult"
23    esac
24 done

```

Рисунок 4.14: Тестовое задание.

Какие(ая) из предложенных ниже инструкций увеличат значение переменной `a` на значение переменной `b` ? Например, если в `a` было записано 10, в `b` было 5, то в `a` должно записаться 15.
Выберите **все подходящие** варианты!

Примечание: если вы планируете проверять варианты ответов у себя в терминале, обратите внимание на то, что содержащие символ `$` тексты могут изменяться при копировании — не забудьте отредактировать их в соответствии с изображением на экране. Это связано с особенностями написания `$` в некоторых видах заданий на Stepik.

Подсказка: обратите особое внимание на кавычки и **пробелы**, они могут как принципиально изменить команду, так и ни на что не повлиять (в зависимости от команды и контекста)!

Выберите все подходящие ответы из списка

☒ Всё правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ `let "a = a + b"`
- ☒ `let "a+=b"`
- ☒ `let "a=$a+$b"`
- ☒ `let a=$a+$b`
- ☒ `let a=a+b`

Рисунок 4.15: Тестовое задание.

В следующем случае срабатывает в первую очередь `pwd`.

Вы можете скачать и изучить скрипт, который мы показали в видеофрагменте: programs.sh.

Пусть вы находитесь в директории `/home/bi/Documents/` и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash
cd /home/bi/
echo "`pwd`"
```

Что в этом случае выведет команда `echo` на экран?

Выберите один вариант из списка

☒ Всё правильно.

- ☐ Код возврата команды `pwd` (0 в случае успешного выполнения и не 0 в случае ошибок)
- ☐ ``pwd``
- ☒ `/home/bi`
- ☐ `pwd`
- ☐ `/home/bi/Documents`

Рисунок 4.16: Тестовое задание.

Следующие тестовые и практические задания не нуждаются в подробном пояснении.

Мы рассказали, что можно проверить код возврата внешней программы прямо в конструкции `if` при помощи `if `program` options arguments` (действия внутри `if` выполнятся, если программа закончилась с кодом 0). Однако это не всегда правда! Если запуск внешней программы выводит что-то в `stdout`, то в проверку `if` поступит именно этот вывод, а не код возврата! Вы можете убедиться в этом, написав простой `bash`-скрипт с использованием, например, `if `pwd``.

Однако как быть, если хочется всё-таки запустить программу `program`, которая пишет что-то в `stdout` и потом выполнить какие-то действия если ее код возврата равен 0? Выберите все верные утверждения или правильно работающие конструкции `if`.

Примечание: во всех вариантах ответов, где есть кавычка, используется именно косая кавычка (```), а не обычная (`"`) или двойная (`"`).

Выберите все подходящие ответы из списка

☒ Абсолютно точно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в комментариях, отвечая на их вопросы, или сравнить своё решение с другими на форуме решений.

- ☒ `if `program` > some_file.txt`
- ☐ Ничего сделать нельзя
- ☐ `if [[ `program` -eq 0 ]]`
- ☒ Сначала запустить `program`, затем `if [[ $? -eq 0 ]]`
- ☐ Сначала `var=`program``, затем `if [[ $var -eq 0 ]]`

Рисунок 4.17: Тестовое задание.

Впишите в форму ниже **строку**, которую выведет на экран команда `echo "counters are $c1 and $c2"` если она находится в скрипте **после десяти вызовов** функции `counter` с параметрами сначала 1, затем 2, затем 3 и т.д., последний вызов с параметром 10.

Подсказка: этот пример можно решить в уме, но если система проверки не принимает ваше решение, то возможно вы что-то упустили (возможно что-то совсем небольшое/невидимое 🤔). В этом случае имеет смысл написать небольшой скрипт на bash, который проделает ровно то, что указано в задании и посимвольно сверить свой ответ с тем, что он выдаст на экран.

Напишите текст

✔ Правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

counters are and 110

Рисунок 4.18: Тестовое задание.

✔ Правильно.

Теперь вам доступен [форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 gcd ()
2 {
3     if [[ -z $M && -z $N ]]; then
4         echo 'bye'
5         exit
6     fi
7
8     if [[ $M -eq $N ]]; then
9         echo "GCD is $M"
10    else
11        if [[ $M -gt $N ]]; then
12            let "M-= $N"
13            gcd $M $N
14        else
15            let "N-= $M"
16            gcd $M $N
17        fi
18    fi
19 }
20
21 while true
22 do
23     read M N
24     gcd $M $N
25 done
```

Рисунок 4.19: Тестовое задание.

Напишите программу. Тестируется через stdin → stdout

✓ Верно.

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 while true
2 do
3   read arg1 smth arg2
4   if [[ $arg1 == "exit" ]]
5   then
6     echo "bye"
7     break
8   elif [[ "$arg1" =~ "^[0-9]+$" && "$arg2" =~ "^[0-9]+$" ]]
9   then
10    echo "error"
11    break
12   else
13     case $smth in
14       "+") let "result = arg1 + arg2";;
15       "-") let "result = arg1 - arg2";;
16       "/" ) let "result = arg1 / arg2";;
17       "*") let "result = arg1 * arg2";;
18       "%") let "result = arg1 % arg2";;
19       "**") let "result = arg1 ** arg2";;
20       *) echo "error" ; break ;;
21     esac
22     echo "$result"
23   fi
24 done
```

Рисунок 4.20: Тестовое задание.

В отличие от name, iname не учитывает регистр.

Пусть в директории /home/bi лежат файлы Star_Wars.avi, star_trek_OST.mp3, STARS.txt, stardust.mpeg, Eddard_Stark_biography.txt .

Отметьте все файлы, которые **найдет** команда `find /home/bi -iname "star*"`, но **НЕ найдет** команда `find /home/bi -name "star*" ?`

Выберите все подходящие ответы из списка

✓ Верно. Так держать!

- ☐ Eddard_Stark_biography.txt
- ☒ STARS.txt
- ☒ Star_Wars.avi
- ☐ stardust.mpeg
- ☐ star_trek_OST.mp3

Рисунок 4.21: Тестовое задание.

Следующие тестовые и практические задания не нуждаются в подробном пояснении.

Задание на понимание работы опций `-path` и `-name` команды `find`. Отметьте **все верные** утверждения из перечисленных ниже.

Выберите все подходящие ответы из списка

☒ Так точно!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ В некоторых случаях `find` с `-name` найдет меньше файлов, чем `find` с таким же запросом, но с `-path`
- ☐ Опции `-path` и `-name` всегда работают одинаково
- ☐ Опция `-path` используется только для поиска директорий, а `-name` только для поиска файлов
- ☐ Опция `-path` аналогична `-name`, но игнорирует размер букв (строчные/прописные) в имени файла
- ☐ Если заменить в команде поиска `-name`, на `-path`, то результат поиска всегда останется неизменным

Рисунок 4.22: Тестовое задание.

Предположим, что в директории `/home/bi/` есть следующая структура файлов и поддиректорий:

```
/home/bi/
├── dir1
│   ├── file1
│   └── dir2
│       ├── file2
│       └── dir3
│           └── file3
```

Какие(ой) из трех файлов (`file1`, `file2`, `file3`) будут найдены по команде `find /home/bi -mindepth 2 -maxdepth 3 -name "file*" ?`

Выберите один вариант из списка

☒ Прекрасный ответ.

- ☐ Только `file3`
- ☐ Все три файла
- ☐ Только `file2`
- ☒ Все кроме `file3`
- ☐ Все кроме `file1`

Рисунок 4.23: Тестовое задание.

Задание на понимание работы опций `-A`, `-B` и `-C` команды `grep`. Пусть у вас есть файл `file.txt` из 10 строк, причем **в каждой строке есть слово "word"**. Если вы выполните на этом файле команды:

```
grep "word" file.txt > results.txt
grep -A 1 "word" file.txt > results.txt
grep -B 1 "word" file.txt > results.txt
grep -C 1 "word" file.txt > results.txt
```

то какая(ие) из них создаст файл `results.txt` наибольшего размера?

Выберите один вариант из списка

☒ Хорошая работа.

- ☐ Все, кроме `grep "word" file.txt > results.txt`
- ☒ `results.txt` будет одинакового размера во всех случаях
- ☐ `grep -A 1 "word" file.txt > results.txt` и `grep -B 1 "word" file.txt > results.txt`
- ☐ `grep -C 1 "word" file.txt > results.txt`
- ☐ `grep -A 1 "word" file.txt > results.txt`

Рисунок 4.24: Тестовое задание.

Предположим, что в файле `text.txt` записаны строки, показанные среди вариантов ответа. Отметьте только те из них, которые выведет на экран команда `grep -E "[xkLXKL]?[uU]buntu$" text.txt`.

Выберите все подходящие ответы из списка

☒ Всё правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ The best OS is Xubuntu
- ☒ I prefer Kubuntu
- ☒ Mac OS X, Windows, Ubuntu
- ☒ Lubuntu is better than Ubuntu
- ☒ Linux is not always Ubuntu
- ☒ Hmm, XKLubuntu

Рисунок 4.25: Тестовое задание.

Что произойдет, если в команде `sed -n "[a-z]*/p" text.txt` не указывать опцию `-n` ?

Выберите один вариант из списка

☒ Абсолютно точно.

- ☒ Каждая строка будет выведена два раза
- ☐ На экран ничего не напечатается
- ☐ Будут выведены все строки файла `text.txt`, в которых есть только большие буквы латинского алфавита
- ☐ Появится сообщение об ошибке

Рисунок 4.26: Тестовое задание.

Напишите текст

☒ Всё получилось!

```
sed 's/[A-Z]\{2,\} /abbreviation /g' input.txt > edited.txt
```

Рисунок 4.27: Тестовое задание.

Какую опцию нужно указать при запуске `gnuplot`, чтобы при его закрытии не были автоматически закрыты и все нарисованные в нём графики?

Выберите один вариант из списка

☒ Отлично!

- ☐ `-raise`
- ☒ `-p, -persist`
- ☐ Графики и так не закрываются автоматически при закрытии `gnuplot`!
- ☐ Такой опции не существует

Рисунок 4.28: Тестовое задание.

Предположим у вас есть файл `data.csv` с двумя столбцами по 10 чисел в каждом. В первой строке не записаны названия столбцов, т.е. ряды данных начинаются прямо с первой строки. Вы запускаете `gnuplot` и вводите в него две команды:

```
set key autotitle columnhead
plot 'data.csv' using 1:2
```

Какое в этом случае будет **название** у построенного **ряда данных** и **сколько** будет нарисовано **точек** на графике?

Выберите один вариант из списка

☒ Правильно, молодец!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Название – первое значение из первого столбца, нарисовано 9 точек (точка из первой строки пропущена)
- ☒ Название – первое значение из второго столбца, нарисовано 9 точек (точка из первой строки пропущена)
- ☐ Название "data.csv" using 1:2, нарисовано 10 точек
- ☐ Название – первое значение из первого столбца, нарисовано 10 точек
- ☐ Название "popame", нарисовано 10 точек

Рисунок 4.29: Тестовое задание.

Напишите текст

☒ Верно.

```
set xtics ("point 1, value ".x1 x1, "point 2, value ".x2 x2, "point3, value".x3 x3)
```

Рисунок 4.30: Тестовое задание.

Напишите текст

☒ Прекрасный ответ.

```
a=a+1
zrot=(zrot+350)%360
set view xrot,zrot
splot -x**2-y**2
pause 0.1
if (a<50) reread
```

Рисунок 4.31: Тестовое задание.

Какая команда(ы) установят файлу `file.txt` права доступа `rwXrw-r--`, если изначально у него были права `r--r--r--`. Укажите **все** **верные** варианты ответа!

Примечание: запись вида `команда1; команда2; команда3` означает, что в терминале последовательно выполнялись все три команды (сначала `команда1`, затем `команда2` и, наконец, `команда3`).

Выберите все подходящие ответы из списка

☒ Хорошая работа.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ `chmod ug+w file.txt; chmod u+x file.txt`
- ☐ `chmod o-wx file.txt; chmod g-x file.txt; chmod a+wx file.txt`
- ☒ `chmod u+wx file.txt; chmod g+w file.txt`
- ☒ `chmod 764 file.txt`
- ☐ `chmod 777 file.txt`
- ☒ `chmod a+wx file.txt; chmod o-wx file.txt; chmod g-x file.txt`

Рисунок 4.32: Тестовое задание.

Выберите все подходящие ответы из списка

☒ Прекрасный ответ.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ `sudo chmod o+w dir`
- ☐ `sudo chown :group dir`
- ☐ `sudo chmod o+x dir`
- ☐ `sudo chmod g+w dir`
- ☒ `sudo chmod a+w dir`
- ☒ `sudo chown user:group dir`

Рисунок 4.33: Тестовое задание.

Отметьте какие характеристики файла можно посчитать с использованием команды `wc`.

Выберите все подходящие ответы из списка

☒ Правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ Количество строк
- ☒ Размер файла в байтах
- ☐ Количество предложений
- ☐ Количество определенных букв (например, количество букв "А")
- ☒ Количество символов

Рисунок 4.34: Тестовое задание.

Впишите в форму ниже команду, которая выведет сколько места на диске занимает текущая директория (при этом **размер** нужно вывести в **удобном для чтения формате** (например, вместо 2048 байт надо выводить 2.0K) и **больше** на экран выводить **ничего не** нужно). В команде указывайте **только необходимые** для выполнения задания **опции и аргументы**, лишних опций указывать не нужно!

Пример: если в текущей директории есть два файла по 800 Кбайт и две поддиректории в каждой из которой лежит по файлу в 400 Кбайт, то загаданная команда должна вывести на экран одно число: 2.4M (также на экране может быть выведен еще и символ ".", обозначающий, что это размер именно текущей директории).

Напишите текст

☒ Хорошая работа.

`du -sh`

Рисунок 4.35: Тестовое задание.

Впишите в форму ниже максимально короткую команду (т.е. в которой минимально возможное число символов), которая позволит создать в текущей директории 3 поддиректории с именами `dir1`, `dir2`, `dir3`.

Если вы придумали команду, которая выполняет эту задачу, а система проверки сообщает вам "Incorrect"/"Неверно", то скорее всего вы придумали не самую короткую команду из возможных!

Напишите текст

✓ Верно. Так держать!

```
mkdir dir{1,2,3}
```

Следующий шаг

Решить снова

Рисунок 4.36: Тестовое задание.

5 Выводы

В рамках данного этапа мы освоили материал 3 этапа курса и выполнили соответствующие тестовые задания.

Список литературы

- Курс «Введение в Linux» (Stepik)